

Inference, Not Just Notation:

Notes on Sampling-Based Key Extraction for Goal Models

Tim Menzies
timm@ieee.org

ABSTRACT

Goal-oriented requirements engineering is rich in notations and comparatively poor in inference. These notes place a small artifact—a ~100-line stochastic interpreter for softgoal graphs plus a delta-debugging key extractor—against that literature. We trace the “requirements engineering equation” from entailment to search, identify what the artifact should be compared against, list the open problems it touches and the functionality it adds, discuss how i^* models relate to use cases (and whether ordering subgoals closes that gap), and assess the value of pairing this inference engine with LLM-based model extraction. All reference links were resolved and verified on 2026-08-16.

1 THE ARTIFACT, IN ONE PARAGRAPH

A goal model is a Python theory: `built <= buy + diy` declares alternatives, `buy <= vendor * breaks(cheap)` a conjunction with contribution links; hard goals gate a query and every softgoal is engaged and labeled in every sampled world. Worlds are drawn ISAMP-style—one guess per choice point, restart instead of back-track, after Crawford and Baker [19]. The best of $n_1=1000$ worlds (by distance to the heaven point of maximal quality satisfaction and minimal leaf purchases) is reduced to a *key*: its settable labels, minus those every world agrees on, minimized by Zeller’s `ddmin` [20] under the test “30 replays with this subset stay within ϵ of the best.” The same interpreter replays seeds prudently (believed goals are settled; disjunctions prefer settled branches). On Horkoff’s seven Kids Help Phone i^* models—the corpus of the SHORT study [18]—keys of 1–13 labels (1–8% of model atoms) steer fresh samples to near the best found world; the whole corpus runs in about eight seconds.

2 THE REQUIREMENTS ENGINEERING EQUATION

The inference story of RE begins with Jackson’s separation of the world and the machine [1] and its crystallization by Zave and Jackson [2]: given specifications S and domain knowledge K , requirements R are met when $S, K \vdash R$. Read literally, the turnstile makes RE a theorem-proving exercise, and much 1990s-era work took up exactly that reading. The rejoinder matured inside the Mylopoulos group: first a revised core ontology of the requirements problem [3], then, in 2010, Techne [4], which recast the problem from *proving* R to *finding* candidate solutions among mandatory and optional goals under preference—search, not deduction. A parallel line had long argued the abductive reading: requirements inference as the generation of assumption sets that explain desired goals [5].

The present artifact is a blunt continuation of that trajectory: it treats the equation stochastically. Solutions are not proved or enumerated; they are *sampled*, the best sample is compressed to the assumptions that matter, and the compressed answer is validated

not by proof but by *replay*—does re-asserting the key reproduce the quality? Where Techne asks “which candidate solutions exist?” this asks “which few commitments regenerate a good one on demand?”

3 REPRESENTATION-RICH, INFERENCE-POOR

The representational lineage is long and healthy: the NFR framework’s softgoals and contribution links [6], i^* ’s strategic actors [7], KAOS’s refinement calculus [8], Tropos and its formal variant [9], the community-standard cleanup of iStar 2.0 [10], lately restated in book form with a fuller worked example [11], and GRL within the User Requirements Notation [12].

Inference over these notations is thinner. The main families:

- *Qualitative label propagation*, forward and backward, from the NFR procedure [6] through its formalization [13];
- *SAT-based* minimum-cost satisfiability over goal graphs [14], later generalized to constrained goal models solved by Optimization Modulo Theories [15];
- *Interactive* evaluation that deliberately keeps a human in the propagation loop [16];
- *Metaheuristic search*: SHORT’s polynomial-time key ranking, evaluated against NSGA-II [18].

Horkoff and Yu’s comparison of satisfaction analysis techniques [17] documents the asymmetry directly: many notations, few analysis procedures, little tooling, and results that rarely travel between notations. The extended systematic mapping of the field [29]—246 top-cited GORE papers—quantifies it: proposals and formalizations dominate, while papers that *evaluate* analyses remain scarce. That imbalance is an argument for cheap, executable baselines: an analysis that runs in seconds on the standard corpus lowers the price of the missing evaluations.

Our niche in that map is *simulation plus test-driven minimization*. The minimization side imports a lineage RE has barely used: `ddmin` [20] and its modern probabilistic and weighted descendants [21, 30], and—closest in spirit—prediction-preserving input minimization, which delta-debug an input to the fragment that keeps an ML model’s answer [22]. We delta-debug an *assumption set* to the fragment that keeps a *stochastic simulator*’s answer; to our knowledge that combination is new in RE.

What to compare against. Four baselines are natural, in increasing strength: (1) SHORT itself, same corpus, same question (done: our keys are 1–8% of atoms against their “often just 12%”); (2) exact optima from minimum-cost SAT [14] or OMT [15] on the same models, giving the optimality gap of sampling; (3) NSGA-II as the standard many-objective yardstick [18]; and (4) interactive evaluation [16] as the human baseline for “how many decisions must a stakeholder actually make?”—which is what a key claims to bound.

4 OPEN PROBLEMS; NEW FUNCTIONALITY

Addressed. (a) *Trade-space summarization*: rather than a Pareto front of hundreds of worlds, the output is a handful of labels a stakeholder can read, audit, and refuse. (b) *Validated recommendations*: keys are tested by replay, so “do these five things” arrives with a measured mean and variance, not a promise. (c) *Luck versus choice*: some models’ best worlds are partly coin fortune from help/hurt links; no key can guarantee them, and the ratio of soft links predicts the gap. Naming that bound—what fraction of observed quality is *decidable at all*—seems to be new. (d) *Scale of machinery*: two files, ~150 lines of code plus seven named constants, eight seconds per corpus; small enough for CI, classrooms, and audit.

Enabled. Seeded what-if analysis (assert any partial belief set, sample the consequences); ablation of stakeholder positions (drop one commitment from a key, watch the quality distribution move); prudence checking as a first-class operation (a claimed optimum that cannot be replayed from its own labels is an artifact—this check caught a real bug in our own port).

5 ISTAR 2.0 AND USE CASES

iStar 2.0 and use cases answer different questions. A goal model is an *intentional variability space*: why actors want things, which alternatives exist, how qualities trade. A use case is a *behavioral contract*: actor–system interaction sequences, main and exceptional flows, pre- and post-conditions. The bridge has been walked in one direction: Santander and Castro derive use cases from i* models [23], and the User Requirements Notation pairs GRL goal models with Use Case Maps precisely because neither view subsumes the other [12].

Does ordering subgoals give use cases? Partly. Our kernel is already closer than it looks: a bare body list runs in order (only the explicit and shuffles), so *sequence is one dialect convention away*, and worlds would become traces. That yields scenario skeletons: ordered, branching (via or), abortable (a failed demand ends the trace). What it does not yield is what use cases are for: actor–system *message* structure, alternative flows selected by *conditions* rather than dice, interruption and resumption, and data. Order is necessary but not sufficient; the URN experience suggests keeping the two views linked rather than merging them. Recent benchmarks of LLM use case recovery [24] suggest the behavioral side may soon be extractable too, which strengthens the two-view position.

6 LLMS AS MODEL EXTRACTORS

Can LLMs read large documents and emit softgoal graphs? The extraction question is already an active subfield: comparative studies of LLMs for goal model extraction [25], extraction from user stories [26], end-to-end generative goal modeling [27], sobering evaluations of prompting limits [28], and extraction of requirements from upstream sources such as online user feedback [34]. The 2025 systematic review of LLMs in RE [31] (74 studies, 2023–24) finds the field GPT-centric, zero- and few-shot, evaluated in controlled settings, and—most relevant here—rarely *integrated into downstream workflows*: the models extract artifacts that nothing then consumes. The formal-methods community has begun sketching the missing loop, a two-way roadmap where LLMs draft formal artifacts and formal tools check them [32, 33]. Extraction *alone* is of limited

value—it produces diagrams that humans must then read, and a 300-node hallucination-prone diagram is not obviously better than the document it summarizes.

The added value appears when extraction feeds an inference engine like this one:

- *Hallucination linting*: undefined or unlinked atoms are mechanical to detect; a graph that cannot be sampled is rejected before anyone reads it.
- *Sanity by simulation*: label histograms over 1000 worlds expose absurd structure (a “quality” that is always satisfied, a hard goal that never is) far faster than visual inspection.
- *Audit by key*: the human reviews 5–13 extracted decisions—each traceable back to source text—instead of the whole graph. The key becomes the interface between the LLM’s recall and the human’s judgment.
- *Robustness signals*: keys stable across seeds and across re-extractions are evidence about the document; keys that wobble localize where the LLM guessed.

The dialect helps here: the model *is* Python (built $\leq$ buy + diy), a friendlier LLM output target than XML interchange formats, and syntax errors fail fast. The risk is unchanged from all LLM pipelines—fluent wrong structure—but a key of ten auditable commitments is a far smaller attack surface than a diagram, and the replay test gives every extracted model a falsifiable claim to fail.

7 CONCLUSION

The field’s imbalance is real: notations accumulated for three decades while inference stayed thin. The artifact described here is one more entry on the thin side—sampling as the solver, delta debugging as the explainer, replay as the validator—cheap enough to run anywhere and small enough to audit. Its natural next experiments are the optimality-gap comparison against exact solvers, a sequence-extended dialect evaluated against derived use cases, and an extract-lint-sample-key pipeline over real documents.

REFERENCES

- [1] M. Jackson. The world and the machine. *ICSE 1995*. <https://doi.org/10.1145/225014.225041>
- [2] P. Zave, M. Jackson. Four dark corners of requirements engineering. *ACM TOSEM* 6(1), 1997. <https://doi.org/10.1145/237432.237434>
- [3] I. Jureta, J. Mylopoulos, S. Faulkner. Revisiting the core ontology and problem in requirements engineering. *RE 2008*. <https://doi.org/10.1109/re.2008.13>
- [4] I. Jureta, A. Borgida, N. Ernst, J. Mylopoulos. Techne: Towards a new generation of requirements modeling languages. *RE 2010*. <https://doi.org/10.1109/re.2010.24>
- [5] T. Menzies. Applications of abduction: knowledge-level modelling. *Int. J. Human-Computer Studies* 45(3), 1996. <https://doi.org/10.1006/ijhc.1996.0054>
- [6] L. Chung, B. Nixon, E. Yu, J. Mylopoulos. *Non-Functional Requirements in Software Engineering*. Springer, 2000. <https://doi.org/10.1007/978-1-4615-5269-7>
- [7] E. Yu. Towards modelling and reasoning support for early-phase requirements engineering. *ISRE 1997*. <https://doi.org/10.1109/isre.1997.566873>
- [8] A. van Lamsweerde. Goal-oriented requirements engineering: a guided tour. *RE 2001*. <https://doi.org/10.1109/isre.2001.948567>
- [9] A. Fuxman, L. Liu, J. Mylopoulos, M. Pistore, M. Roveri, P. Traverso. Specifying and analyzing early requirements in Tropos. *Requirements Engineering* 9(2), 2004. <https://doi.org/10.1007/s00766-004-0191-7>
- [10] F. Dalpiaz, X. Franch, J. Horkoff. iStar 2.0 language guide, 2016. <https://arxiv.org/abs/1605.07767>
- [11] F. Dalpiaz, X. Franch, J. Horkoff. Definition and use of iStar 2.0. In *Social Modeling Using the i* Framework*, Springer, 2024. https://doi.org/10.1007/978-3-031-72107-6_5
- [12] D. Amyot, S. Ghanavati, J. Horkoff, G. Mussbacher, L. Peyton, E. Yu. Evaluating goal models within the goal-oriented requirement language. *Int. J. Intelligent Systems* 25(8), 2010. <https://doi.org/10.1002/int.20433>

- [13] P. Giorgini, J. Mylopoulos, E. Nicchiarelli, R. Sebastiani. Reasoning with goal models. *ER 2002*. https://doi.org/10.1007/3-540-45816-6_22
- [14] R. Sebastiani, P. Giorgini, J. Mylopoulos. Simple and minimum-cost satisfiability for goal models. *CAiSE 2004*. https://doi.org/10.1007/978-3-540-25975-6_4
- [15] C. M. Nguyen, R. Sebastiani, P. Giorgini, J. Mylopoulos. Multi-objective reasoning with constrained goal models. *Requirements Engineering* 23, 2018. <https://doi.org/10.1007/s00766-016-0263-5>
- [16] J. Horkoff, E. Yu. Interactive goal model analysis for early requirements engineering. *Requirements Engineering* 21, 2016. <https://doi.org/10.1007/s00766-014-0209-8>
- [17] J. Horkoff, E. Yu. Comparison and evaluation of goal-oriented satisfaction analysis techniques. *Requirements Engineering* 18, 2013. <https://doi.org/10.1007/s00766-011-0143-y>
- [18] G. Mathew, T. Menzies, N. Ernst, J. Klein. SHORT: shortcutting the search for solutions in goal models, 2017. <https://arxiv.org/abs/1702.05568>
- [19] J. Crawford, A. Baker. Experimental results on the application of satisfiability algorithms to scheduling problems. *AAAI 1994*. <https://cdn.aaai.org/AAAI/1994/AAAI94-168.pdf>
- [20] A. Zeller, R. Hildebrandt. Simplifying and isolating failure-inducing input. *IEEE TSE* 28(2), 2002. <https://doi.org/10.1109/32.988498>
- [21] M. Zhang et al. Toward a better understanding of probabilistic delta debugging, 2024. <https://arxiv.org/abs/2408.04735>
- [22] S. Suneja, Y. Zheng, Y. Zhuang, J. Laredo, A. Morari. Probing model signal-awareness via prediction-preserving input minimization, 2020. <https://arxiv.org/abs/2011.14934>
- [23] V. Santander, J. Castro. Deriving use cases from organizational modeling. *RE 2002*. <https://doi.org/10.1109/icre.2002.1048503>
- [24] UCRBench: benchmarking LLMs on use case recovery, 2025. <https://arxiv.org/abs/2512.13360>
- [25] A comparative study of large language models for goal model extraction. *MOD-ELS Companion (SAM) 2024*. <https://doi.org/10.1145/3652620.3686246>
- [26] Goal model extraction from user stories using large language models. Springer LNCS, 2024. https://doi.org/10.1007/978-3-031-70245-7_19
- [27] Generative goal modeling, 2025. <https://arxiv.org/abs/2509.01048>
- [28] Evaluating LLM-based goal extraction in requirements engineering: prompting strategies and their limitations, 2026. <https://arxiv.org/abs/2604.22207>
- [29] J. Horkoff, F. Aydemir, E. Cardoso, T. Li, A. Maté, E. Paja, M. Salnitri, L. Piras, J. Mylopoulos, P. Giorgini. Goal-oriented requirements engineering: an extended systematic mapping study. *Requirements Engineering* 24, 2019. <https://doi.org/10.1007/s00766-017-0280-z>
- [30] R. Wang et al. WDD: weighted delta debugging. *ICSE 2025*. <https://doi.org/10.1109/ICSE55347.2025.00071>
- [31] Large language models (LLMs) for requirements engineering (RE): a systematic literature review, 2025. <https://arxiv.org/abs/2509.11446>
- [32] Formal requirements engineering and large language models: a two-way roadmap. *Information and Software Technology* 181, 2025. <https://doi.org/10.1016/j.infsof.2025.107697>
- [33] A short survey on formalising software requirements using large language models, 2025. <https://arxiv.org/abs/2506.11874>
- [34] From online user feedback to requirements: evaluating large language models, 2025. <https://arxiv.org/abs/2510.23055>
- [35] L. Zadeh. Fuzzy sets. *Information and Control* 8(3), 1965. [https://doi.org/10.1016/S0019-9958\(65\)90241-X](https://doi.org/10.1016/S0019-9958(65)90241-X)